

# Как это устроено изнутри: ТСПУ, сертификаты Минцифры и белые списки

---

Черновик. Материал технический и рассчитан на тех, кому мало ответа «ну, заблокировали».

В [прошлой статье](#) я честно написал, что главная проблема нашей схемы — белые списки, и что решения у меня нет. Здесь я разбираюсь, почему решения не было, и предлагаю архитектуру, в которой оно появляется.

Три темы — ТСПУ, сертификаты и белые списки — обычно обсуждают порознь. Это ошибка: они собираются в одну конструкцию. Блокировки решают, **куда нельзя**. Белые списки решают, **куда можно** — и это принципиально другой режим. Сертификаты решают третью задачу: **видеть, что внутри**. Каждый слой закрывает дыру, оставленную предыдущим, и обходить их поодиночке бессмысленно.

Где могу — привожу собственные замеры, а не пересказ. Где не могу — прямо пишу, что это чужие данные и насколько я им верю.

---

## 1. ТСПУ: что это, что режется и как это обойти

---

### Что это физически

ТСПУ — «технические средства противодействия угрозам». Железо, которое по закону о суверенном интернете (2019) устанавливается у операторов связи, но управляется централизованно, не оператором. Оператор обязан пропустить через него трафик и не имеет права смотреть, какие там правила.

Ключевое для понимания: **ТСПУ стоит на границе оператора**, а не у вас дома и не на сервере. Поэтому один и тот же сайт может открываться с домашнего вайфая и не открываться с мобильного — это разные операторы и разные коробки с разными наборами правил. И поэтому же бессмысленны утверждения вида «в России заблокировано то-то»: правильный вопрос — «заблокировано у какого оператора, в каком регионе и на этой ли неделе».

### На что оно смотрит

Три уровня, по возрастанию сложности:

**Адреса и порты.** Самое дешёвое: списки IP и подсетей. Дёшево, грубо, ловит заодно кучу постороннего.

**Содержимое рукопожатия.** Когда вы открываете HTTPS-сайт, самый первый пакет — `ClientHello` — уходит **открытым текстом**, шифрование ещё не началось. В нём лежит поле SNI: имя сайта, куда вы идёте. Это единственная дырка в HTTPS, и именно через неё смотрят все.

Там же виден «отпечаток» вашей TLS-библиотеки — порядок полей, набор шифров, расширения. По нему отличают Chrome от curl и от ВПН-клиента. Называется JA3 или JA4.

**Поведение.** Размеры первых пакетов, паузы между ними, число одновременных соединений к одному адресу. Здесь протокол уже не важен — важно, как трафик себя ведёт.

### Что оно делает

Три действия, и различать их важнее, чем сам факт неработоспособности:

- **Дроп** — пакет молча выбрасывается. Клиент ничего не понимает и висит в таймауте. Самое неприятное для диагностики.
- **RST** — соединение убивается подделанным TCP-пакетом «сброс». Выглядит как «сервер закрыл соединение».
- **Замедление** — трафик пропускается, но душится. Так в своё время душили Twitter, так сейчас душат YouTube.

## Что я замерил сам

Это первые руки, не пересказ. Подопытный — маршрут с моего узла в Selectel на узел в Амстердаме, единственный, который упорно не работал. Раньше я списал его как «Selectel режет всю подсеть» — и это оказалось **неверно**.

**Действие — дроп ровно одного пакета.** Я снял захват трафика одновременно на обоих концах. Рукопожатие TCP проходит целиком и видно с обеих сторон. А следом сегмент с `ClientHello` и все пять его повторов до Амстердама **не доезжают вообще**. Не отказ, не сброс — исчезновение. Сервер честно ждёт и через десять секунд закрывается сам.

**Признак — несовпадение имени с адресом.** Решающий опыт: одно и то же имя, но на разные адреса.

| Имя в <code>ClientHello</code> | Куда идём                                       | Результат |
|--------------------------------|-------------------------------------------------|-----------|
| <code>www.google.com</code>    | <code>142.251.157.119</code> — настоящий Google | 33 мс     |
| <code>github.com</code>        | <code>140.82.121.4</code> — настоящий GitHub    | 86 мс     |
| <code>vk.com</code>            | <code>87.240.132.78</code> — настоящий VK       | 19 мс     |
| те же три имени                | <code>109.104.153.242</code> — мой сервер       | дроп      |

Вывод однозначный: блокируются **не домены**. Среди них `github.com`, `vk.com`, `mail.ru` — их в России никто не блокирует. Ловится сам факт, что известное имя надето на чужой адрес.

Это называется детект подмены SNI. И это ровно то, на чём построен REALITY.

**Как устроен сопоставитель имён.** Суффиксный, по границам меток: режется `google.com` и любой поддомен, включая несуществующий `xyzabc.google.com`; проходят `notgoogle.com`, `google.com.tr`, `google.com.evil.net`. То есть разбор имени сделан правильно.

А вот дальше — грубая ошибка реализации. Сравнение **побайтовое**, хотя доменные имена регистронезависимы по стандарту (RFC 4343). Проходят:

`GOOGLE.COM`      `Www.Google.Com`      `vk.com.` (с точкой в конце)

Я проверил это не на бумаге. Добавил на сервер второе написание имени и снял A/B прямо на заблокированном узле — всё побайтово одинаково, кроме регистра:

```
serverName = www.google.com → таймаут
serverName = WWW.GOOGLE.COM → работает, 44 мс, выход в Амстердаме
```

Маршрут, который был объявлен безнадёжным, поднялся сменой регистра одной буквы.

**Что не помогает.** Тоже полезно знать:

- **Порт не важен вообще.** 443, 8443, 2053, 8080 и даже 60000 — поведение одинаковое. Расхожее «высокие порты инспектируются поверхностно» здесь просто неверно.
- **Фрагментация не спасает.** Я резал `ClientHello` на куски по 100, 64, 40, 16 и 8 байт — фильтр пересобирает поток и всё равно находит имя. Многие «обходилки» построены именно на разрезании первого пакета; против этой коробки они бесполезны.
- **Частота ни при чём.** Популярная версия — «блокируют больше трёх параллельных рукопожатий» — у меня **не подтвердилась**: пробы шли строго по одной, с паузами в четыре секунды и в перемешанном порядке, и результат воспроизводился идеально.

**Где стоит.** Первые четыре маршрутизатора — сеть самого Selectel, до них обе пробы доходят одинаково. Дальше маршрутизаторы на истёкший TTL не отвечают вовсе, поэтому получается не точка, а промежуток: съездается на пятом-шестом хопе, то есть **на выходе из хостинга в магистраль**.

**Насколько узко включено.** Важная деталь: правило действует только для пары «мой узел в Selectel → этот конкретный адрес». Та же подмена имени с того же узла на 1.1.1.1, 8.8.8.8 и на мои же другие серверы проходит свободно. На тот же адрес из Timeweb, из VPSville и из домашней сети — тоже проходит. То есть это не глобальное правило, а **точечно включённая проверка**. Так и работает: сначала адрес чем-то привлекает внимание, потом на него навешивают инспекцию.

## Оговорка про мои замеры

Я тыкался из сети, где сижу постоянно, и из российских дата-центров. Это **не** репрезентативная выборка. Крупный корпоративный оператор вполне может фильтроваться мягче — не по злему умыслу, а потому что у него другой класс подключения и другие настройки. Мобильные сети режут заметно агрессивнее, и разброс между регионами там большой.

Причём «мягче» не значит «никак». Прогнав по этой самой сети общий зонд, я сразу увидел две вещи, которых не ждал:

- **youtube.com не разрешается в адрес вообще.** Не блокировка соединения, а обрыв на самом первом шаге: имя не превращается в адрес. Это уже уровень DNS, ещё до всякого ТСПУ.
- **1.1.1.1 мёртв полностью** — ни одного ответа ни на один запрос. При этом 8.8.8.8, 9.9.9.9 и Яндекс отвечают штатно. То же самое, кстати, независимо описывают в замерах на мобильных операторах: публичный резолвер Cloudflare закрывают в первую очередь.

То есть даже «лояльный» канал — не чистый. Просто закрыто другое и другими средствами.

Поэтому раздел будет дополнен: нужна серия замеров из разных сетей — домашний вайфай, корпоративный, отельный, у друзей, и главное — мобильный интернет в Москве против мобильного в области. Методика и инструменты для этого описаны в разделе 5.

## Как это обходить

Логика простая: фильтр наказывает за **расхождение** имени и адреса. Значит, надо перестать расходиться.

**Плохое решение — подобрать имя, которого фильтр не знает.** Работает прямо сейчас, делается за пятнадцать минут. Но это гонка: справочник имён пополняется, и однажды ваше имя туда попадёт — молча, а вы узнаете, когда всё встанет.

**Плохое решение — регистр.** Тоже работает прямо сейчас. Тоже временно: чинится одной строкой у них.

**Хорошее решение — своё имя на своём адресе.** Покупаете домен (пять-десять евро в год), направляете его на свой сервер, выпускаете нормальный сертификат Let's Encrypt. Теперь имя в ClientHello **действительно принадлежит** этому адресу. Проверка проходит честно, потому что обманывать нечего.

И самое приятное: **REALITY при этом можно оставить**. Достаточно поднять на том же сервере настоящий сайт на своём домене и указать его целью REALITY вместо `www.google.com`. Тогда:

- SNI — ваш домен, адрес — ваш сервер, сверка сходится;
- анти-зондирование сохраняется: если кто-то придёт проверять, ему ответит настоящий сайт с настоящим сертификатом;
- вы больше не притворяетесь Гуглом, а значит и попасться на притворстве не можете.

Теряется одно: трафик перестаёт выглядеть как поход на большой известный сайт и выглядит как поход на маленький неизвестный. По нынешним правилам это выгодный размен — за «маленький неизвестный» пока не наказывают, а за подделку крупного уже наказывают.

Альтернатива, если хочется ещё и прятать сам адрес сервера — VLESS поверх настоящего TLS с транспортом ХНТТР или gRPC. Такое можно спрятать за CDN, и тогда снаружи виден адрес CDN, а не ваш. Минус: на мобильных операторах с белыми списками CDN обычно не помогает (об этом в разделе 3).

---

## 2. SSL, сертификаты Минцифры и что вообще происходит

---

### Что такое сертификат, если совсем просто

Когда вы открываете `https://`, происходят две разные вещи, и их всё время путают.

**Первое — шифрование.** Канал между вашим браузером и сервером закрывается так, что посередине никто не прочитает содержимое. Это делается автоматически и ни от каких сертификатов не зависит.

**Второе — подтверждение личности.** А откуда браузер знает, что на том конце действительно `sberbank.ru`, а не кто-то, кто перехватил соединение? Вот для этого и нужен сертификат.

Сертификат — это electronic-удостоверение: «предъявитель сего действительно владеет доменом `sberbank.ru`». Подписано оно **удостоверяющим центром** — организацией, которая перед выдачей проверила, что заявитель правда владеет доменом.

Дальше самое важное. В вашем браузере (и в операционной системе) зашит список удостоверяющих центров, которым он доверяет **по умолчанию** — сотня-другая организаций по всему миру. Это называется корневым хранилищем. Если сертификат подписан кем-то из списка — замочек зелёный. Если кем-то посторонним — красное предупреждение на весь экран.

Вся конструкция держится на одном допущении: центры из списка **не выдают сертификаты кому попало**. За этим следят, за нарушения из списка выкидывают навсегда.

### Что такое сертификат Минцифры

Минцифры создало свой удостоверяющий центр — НУЦ, корневой сертификат называется Russian Trusted Root CA. Технически это совершенно обычный удостоверяющий центр, ничего экзотического.

Разница одна, но решающая: **его нет в корневых хранилищах** Chrome, Firefox, Safari и Edge. И не будет — для попадания туда нужно пройти международный аудит и соответствовать требованиям, которые в текущей ситуации несовместимы с тем, как этот центр устроен.

Поэтому сайт с таким сертификатом в обычном браузере даёт предупреждение об опасности. Чтобы предупреждение исчезло, пользователь должен **вручную установить корневой сертификат** в свою систему. Либо поставить браузер, где он уже вшит — Яндекс Браузер или «Атом».

### Почему это происходит сейчас

Это не прихоть, а следствие. Западные удостоверяющие центры находятся под своими юрисдикциями и обязаны исполнять санкции. В июне 2026 японский GlobalSign начал **отзывать** сертификаты российских доменов — оценки говорят о пятнадцати-двадцати тысячах.

Отозванный сертификат — это мгновенно неработающий сайт: у всех посетителей красный экран. Что характерно, большинство пострадавших сначала ушли не в НУЦ, а к греческому HARICA и в Let's Encrypt. Но по мере того как закрывались и эти каналы, к августу 2026 крупные банки перешли на НУЦ уже вынужденно.

То есть переезд gu-контура на российские сертификаты — не заговор и не инициатива снизу. Это то, что остаётся, когда остальные двери закрываются одна за другой.

## Чем это опасно и как связано с блокировками

А вот здесь начинается интересное, и связь с первым разделом прямая.

Вспомните, как всё держится: браузер верит любому сертификату, подписанному центром из списка. Если вы добавили в список ещё один центр, то этот центр может выпустить **валидный сертификат на любой сайт в мире**. На `google.com`. На ваш банк. На что угодно.

А теперь соедините это с первым разделом. На границе оператора уже стоит железо, через которое идёт весь ваш трафик. Если это железо получит сертификат, которому ваш браузер доверяет, оно сможет встать посередине вашего HTTPS-соединения: расшифровать, прочитать, зашифровать заново и отправить дальше. Замочек останется зелёным. Вы ничего не заметите.

Это не конспирология и не гипотеза о намерениях — это простое описание того, какая техническая возможность появляется. Ровно поэтому корневые хранилища и охраняются так строго.

Отсюда вся конструкция и складывается в одно целое:

- **блокировки** — решают, куда нельзя;
- **белые списки** — решают, куда можно, всё остальное закрыто;
- **свой корневой сертификат** — снимает последнее ограничение: даёт возможность видеть, что внутри.

## Что с этим делать практически

**Не устанавливайте корневой сертификат в систему.** Ни на макбук, ни на айфон, ни на рабочий компьютер. Установка в систему — это про весь трафик всех приложений сразу.

**Если какой-то сайт без него не открывается** — заведите отдельный браузер. Яндекс Браузер или «Атом», где он уже вшит, и пользуйтесь им **только** для этого сайта. Держите там госуслуги и банк, и больше ничего. Идеально — отдельным профилем или вообще на отдельном устройстве.

**Не кликайте «всё равно перейти»** на предупреждении, если собираетесь вводить пароль. Предупреждение может быть и следствием реальной подмены.

**И важное, чтобы не было иллюзий: ВПН эту проблему не решает.** Совсем. TLS-соединение устанавливается между вашим браузером и сайтом, туннель его только переносит и в него не заглядывает. Если сайт предъявляет сертификат, которому вы не доверяете, — вы не будете доверять ему и через туннель.

Что ВПН здесь действительно делает — даёт российский адрес выхода. Многие банки и госсервисы просто не пускают с зарубежных IP. Ради этого в архитектуре из раздела 4 российский контур ходит напрямую с российского узла, а не через границу.

---

## 3. Белые списки: самое сложное

---

### Что это

Обычная блокировка — это чёрный список: запрещено перечисленное, остальное можно. Белый список — наоборот: **разрешено только перечисленное, остальное закрыто**.

Разница не количественная, а качественная. При блокировках у вас работает интернет с дырками. При белых списках у вас нет интернета — есть доступ к нескольким сотням российских сервисов. Не открывается `google.com`. Не работает ни один ВПН из тех, что мне известны, включая мой.

Включают это в основном на **мобильном интернете**, точно по времени и месту. По открытым данным, к марту 2026 режим применялся в 68–71 регионе. Чем южнее регион, тем чаще. Домашний проводной интернет пока

задевает редко — но статьи на Хабре про «белые списки на домашнем интернете» уже появились, так что рассчитывать на это как на константу не стоит.

### Как устроено технически

Здесь я обязан быть аккуратным: своих замеров у меня нет — при белых списках у меня нет и связи, чтобы их снять. Всё ниже — сведение чужих наблюдений, и они **между собой расходятся**. Само расхождение информативно, поэтому привожу его как есть.

**Уровень адресов.** Маршрутизируются только разрешённые подсети, остальные пакеты умирают на роутере. По результатам сканирований — порядка 63 тысяч разрешённых адресов в 2557 автономных системах. Примерно пятая часть всех разрешённых адресов приходится на Яндекс.Облако.

**Уровень имён.** Проверяется SNI в рукопожатии TLS. Из миллиона доменов рейтинга Tranco разрешёнными оказались около тысячи — то есть примерно 0,1% интернета.

**«Отсечка в 16 килобайт».** Самая любопытная деталь. TCP-соединение устанавливается, проходит десять-четыренадцать пакетов, около 16–20 КБ — и дальше тишина. Соединение не рвётся, просто перестаёт передавать. На практике это выглядит как «страница начала грузиться и висит»: HTML успел прийти, а стили, скрипты и запросы к API — уже нет. Придумано, судя по всему, чтобы не выглядеть как поломка сети.

**Главное противоречие.** Достаточно ли подставить разрешённое имя, или нужен ещё и разрешённый адрес?

- Автор разбора «отсечки» утверждает: достаточно SNI. Он подставлял `ok.ru` в рукопожатие к заблокированному адресу и получал нормальные 55 МБ/с вместо обрыва на 16 КБ.
- Сообщество, собирающее списки, пишет прямо противоположное: «даже если вы идеально подделали разрешённый домен, соединение заблокируют, если адрес вашего сервера не в списке».
- Замеры на Билайне: «любой SNI работает на адресе Яндекса, но никакой SNI не работает на моём VPS».

Это не значит, что кто-то врёт. Это значит, что **режимы разные у разных операторов, регионов и даже базовых станций**, и старые «SNI-only» установки соседствуют с новыми «IP + SNI». Практический вывод: единственного правильного ответа нет, надо мерить на месте. Отсюда и весь раздел 5.

### Почему наша нынешняя схема не выживает

Теперь понятно, почему. Наш российский узел — обычный VPS у обычного хостера, его адреса в белом списке нет. Имя, которым мы прикрываемся — `www.yandex.ru`, оно-то как раз в списке. То есть:

- в режиме «только SNI» схема **могла бы** выжить;
- в режиме «IP + SNI» она мертва гарантированно.

А на практике она не выживает — значит, там, где я проверял, работает второй режим. Или третий, о котором я не знаю.

### Что придумали другие и что из этого живо

Свожу известные подходы вместе с их состоянием. Даты важнее самих методов: это область, где полугодовая инструкция скорее вредна.

| Подход                       | Идея                                                               | Состояние                                                                                        |
|------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Разрешённый SNI на своём VPS | Подставить <code>ok.ru</code> или <code>ya.ru</code> в рукопожатие | Живёт там, где проверяется только имя. Мертво при связке «IP + SNI»                              |
| Ретранслятор в Яндекс.Облаке | Свой узел внутри разрешённых адресов                               | Работало, потом сломалось: облако и сам Яндекс — <b>разные</b> автономные системы, адреса облака |



| Подход                     | Идея                                                 | Состояние                                                                                                     |
|----------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
|                            |                                                      | вычистили из списков. К апрелю 2026 из рекомендаций убрано                                                    |
| Serverless в Яндекс.Облаке | Функции и API Gateway как прокси                     | API Gateway закрыли. Функции частично живы, но лимиты бесплатного тарифа делают это игрушкой                  |
| WebRTC через видеозвонки   | Туннель внутри звонка в Яндекс.Телемосте             | Работает в принципе; сообщения до 8 КБ, требует нарезки, годится на «последний рубеж», а не на повседневность |
| Туннель в DNS              | Данные внутри DNS-запросов, где UDP/53 наружу открыт | Живёт там, где не закрыли сторонние резолверы. Медленно, но чинит именно катастрофу                           |
| Ретрансляторы TURN         | Публичные серверы Яндекса и VK для видеозвонков      | Активно закрывают                                                                                             |
| CDN (Cloudflare)           | Прятаться за адресами CDN                            | На домашнем интернете помогает, <b>на мобильном с белыми списками — нет</b> : Cloudflare в списки не входит   |

Общая картина безрадостная и честная: **надёжного, стабильного и простого обхода белых списков сегодня не существует**. Всё, что работает, работает временно и в конкретном месте. Кто обещает иное — либо продаёт, либо не проверял.

### Из чего может состоять решение

Раз надёжного одного нет, стройте так, чтобы не зависеть от одного. Три принципа:

**Первый: быть внутри разрешённого адресного пространства.** Это единственный подход, который бьёт в корень, а не в симптом. Нужен вход, чьи адреса в белом списке. Аренда у российского облачного провайдера, чьи диапазоны туда входят — самое очевидное. Уязвимость известна: как только конкретный диапазон становится популярным у обходчиков, его вычищают, что и случилось с Яндекс.Облаком.

**Второй: несколько разных входов и автоматический выбор живого.** Не «резервный конфиг, который надо руками включить», а группа входов, из которой клиент сам берёт работающий. Это же чинит и обычные блокировки: сгорел один адрес — переехали на другой, вы не заметили.

**Третий: аварийный канал на другой физике.** Туннель в DNS или WebRTC — медленно, зато переживает то, что убивает обычный TLS. Не для просмотра видео, а для «списаться и понять, что происходит».

Конкретика — в следующем разделе.

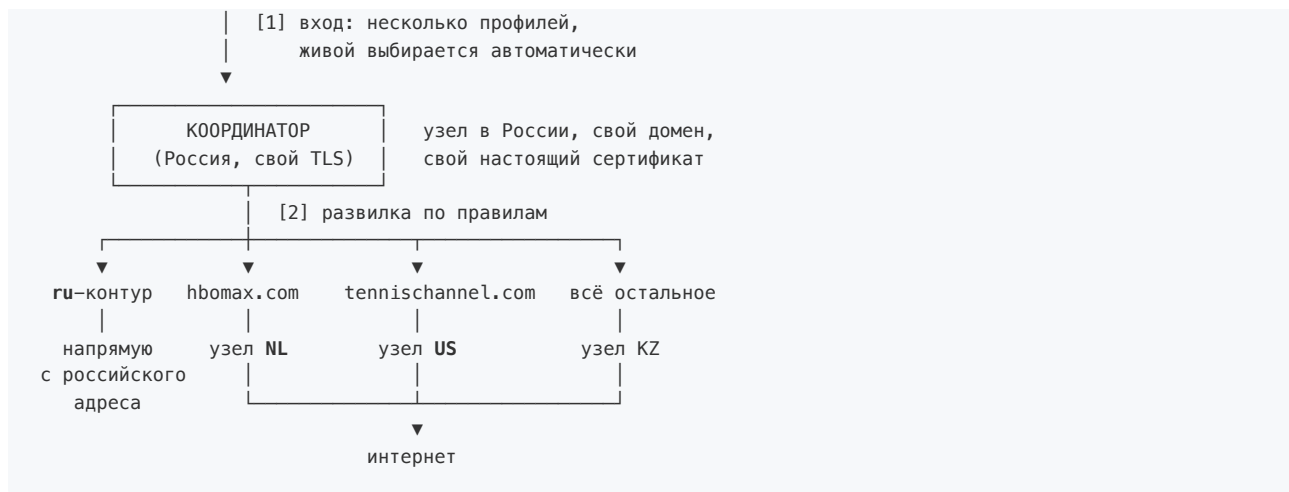
---

## 4. Целевая архитектура

Чего хочется по итогу: один тумблер. Включил — и всё работает, быстро, стабильно и в том числе при белых списках. Никаких «попробуй другой профиль».

### Схема

iPhone / Mac / ПК / что угодно



## Участок 1: клиент → координатор

Самый уязвимый участок — именно он идёт через ТСПУ вашего оператора. Здесь нужен не один способ входа, а несколько, объединённых в группу.

**Профиль А, повседневный: REALITY на своём домене.** На координаторе поднимается настоящий сайт на вашем домене с сертификатом Let's Encrypt, и он же указывается целью REALITY. Имя в рукопожатии — ваш домен, адрес — ваш сервер, сверка из раздела 1 проходит честно. Анти-зондирование сохраняется: любопытному ответит настоящий сайт.

**Профиль Б, обходной: VLESS + XHTTP поверх настоящего TLS.** Тот же домен, другой транспорт. Смысл — в возможности спрятаться за CDN, если адрес координатора однажды попадёт в блок. Помогает от блокировки по адресу, не помогает от белых списков.

**Профиль В, для белых списков: вход внутри разрешённого адресного пространства.** Отдельный маленький узел у провайдера, чьи адреса в белом списке, с разрешённым именем в SNI. Это профиль-нахлебник: он нужен несколько дней в году, но именно он закрывает сценарий, который сейчас не закрыт ничем.

**Профиль Г, аварийный: туннель в DNS.** Килобайты в секунду. Не для пользования, а для того, чтобы в отрезанном регионе оставаться на связи.

Клиент — [sing-box](#): есть под iOS, macOS, Windows, Linux и Android, умеет группу `urldtest`, которая сама опрашивает профили и выбирает живой. Вот вам и один тумблер: пользователь видит один переключатель, за которым группа.

Ключи раздаются [ссылкой на подписку](#), а не файлом. Тогда добавление нового профиля или замена сгоревшего адреса не требует от пользователя вообще ничего.

## Участок 2: развилка на координаторе

Правила маршрутизации живут **на координаторе**, а не в клиенте. Это принципиально: одно место правки, и все устройства всех пользователей получают изменение сразу.

Три вида назначения:

**Российский контур — напрямую.** Банки, госуслуги, маркетплейсы, российские кинотеатры. Выход с российского адреса координатора. Заодно уходят плашки «кажется, у вас ВПН» и перестают ломаться приложения, которые обижаются на зарубежный адрес.

**Настраиваемая маршрутизация по доменам.** `hbomax.com` → узел в Нидерландах, `tennischannel.com` → узел в США, и так далее — обычным списком в конфигурации:



```
hbomax.com, max.com      → nl
tennischannel.com, espn.com → us
*.ru, банки, госуслуги   → напрямую
всё остальное             → kz
```

Всё остальное — в узел по умолчанию.

Технически это `routing.rules` в Xray или sing-box: правило по домену указывает на исходящий тег. Определение домена работает даже внутри туннеля, потому что координатор видит SNI приходящих соединений.

### Участок 3: координатор → зарубежные узлы

Здесь проще: это связь между двумя серверами, и на неё смотрят куда меньше. REALITY подходит, но и здесь лучше на своём домене — по той же причине, по которой у меня сегодня отвалился один маршрут.

Узлов несколько, по одному на страну выхода. Плюс автоматическая проверка живости, чтобы упавший выбывал сам.

### Что это стоит

- домен — 5–10 € в год, один на всё;
- координатор в России — уже есть;
- зарубежные узлы — уже есть, по одному на страну;
- узел для белых списков — 400–500 ₽ в месяц, и то по потребности;
- сертификаты — бесплатно.

То есть почти всё уже куплено. Не хватает домена и одного маленького узла.

### Что честно не решено

- **Профиль В — гипотеза.** Пока не проверено на месте, будет ли конкретный провайдер в белом списке и надолго ли. Это первое, что надо мерить.
- **Аварийный канал** годится для текста, не для жизни.
- **Диапазоны вычищают.** Любой вход внутри разрешённого пространства — расходник. Архитектура должна позволять менять его без переделки всего.

---

## 5. Как это мерить

---

Дальше — план, потому что без замеров всё выше остаётся набором правдоподобных предположений.

### Серия по сетям

Один и тот же набор проб, прогнанный из разных сетей. Проверяем ровно то, что я нашёл в разделе 1: смотрит ли фильтр на связку «имя ⇔ адрес», и где это включено.

Сети: домашний вайфай, корпоративный, отельный, у друзей и — главное — **мобильный интернет, в столице и в области**. Гипотеза, которую проверяем: корпоративные и домашние каналы фильтруются мягче мобильных, а мобильные различаются между регионами.

Инструмент — `scripts/netprobe.py`. Запускается без установки чего-либо, один файл:

```
python3 scripts/netprobe.py мой_сервер=IP --cases tls12:www.google.com,tls12:example.com
```

Если первое молчит, а второе отвечает — сверка «имя ↔ адрес» включена и в этой сети.

## Поездка в область: замер под белыми списками

Здесь проблема методологическая: в момент, когда белые списки включены, связи со мной нет, и подсказать я ничего не смогу. Значит, инструмент должен работать полностью сам, ничего не спрашивать и складывать сырые данные на диск.

Для этого — комплект из двух частей.

**Маяк** ( `scripts/blackout_beacon.py` ) заранее ставится на серверы. Слушает много портов сразу, по TCP и UDP, отвечает опознавательным маркером и **записывает в журнал всё, что до него долетело**. Смысл этой второй половины: даже если ответ до вас не дошёл, запись в журнале маяка доказывает, что ваш пакет **вышел** — а это совсем другой диагноз, чем «не вышел».

**Зонд** ( `scripts/blackout_probe.py` ) запускается на ноутбуке в отрезанном регионе. Ничего не спрашивает, работает по кругу и пишет всё в файл. Что он выясняет:

1. **Работает ли DNS вообще** и какой. Резолвер оператора, `8.8.8.8` , `1.1.1.1` , `77.88.8.8` , DoH. Подменяются ли ответы.
2. **Что открывается из разрешённого** — берётся список из сообщества, проверяется реально.
3. **На каком уровне закрыто остальное**. Та же методика, что в разделе 1: не резолвится (DNS) / не устанавливается TCP (адреса) / TCP есть, а рукопожатие пропадает (SNI).
4. **Главный вопрос: SNI или адрес**. Разрешённое имя на мой неразрешённый адрес. Пройдёт — режим «только имя», и профиль с подставным SNI спасает. Не пройдёт — нужен вход внутри разрешённых адресов, и никак иначе.
5. **Есть ли отсечка в 16 КБ**. Маяк по запросу отдаёт мегабайт, зонд считает, сколько дошло. Если обрыв на 16–20 КБ — та самая механика.
6. **Что живо из другой физики**. UDP на разные порты, QUIC, ICMP, трассировка.
7. **Живы ли мои нынешние маршруты** — все ключи прогоняются по очереди.

Возвращаетесь в нормальную сеть — я забираю журнал зонда, журналы маяков с серверов, свожу и получаю точный ответ, какой профиль строить.

Скрипты лежат в `scripts/` , подробности запуска — в `scripts/README.md` .

---

## Что дальше

Порядок действий по итогам всего разобранного:

1. **Купить домен**. Разблокирует и профиль А, и профиль Б, и снимает проблему из раздела 1 по-настоящему, а не заплаткой.
  2. **Перевести вход на REALITY со своим доменом**. Перестаём притворяться Гуглом.
  3. **Собрать серию замеров по сетям** — дополнить раздел 1 фактами вместо одной точки наблюдения.
  4. **Съездить и снять данные под белыми списками**. Без этого профиль В строить не на чем.
  5. **Собрать координатор с развилкой** и настраиваемой маршрутизацией по доменам.
  6. **Перевести клиентов на подписку** и группу с автовыбором — тот самый один тумблер.
-

## Источники

---

Свои замеры отмечены в тексте отдельно; всё остальное — отсюда.

### ТСПУ и блокировки

- [Заморозка по fingerprint: как ТСПУ ломает соединения по поведению, а не по протоколу](#) — Хабр
- [О схеме ограничений РКН в июне 2026-го](#) — Хабр
- [Как работает DPI и ТСПУ в 2026](#)
- [Russia's internet censorship in 2026](#) — Zona
- [Access Denied: How the Kremlin controls the internet](#) — отчёт ФБК
- [Camouflage and Routing by Domain via SNI Fallback](#) — документация Project X

### Сертификаты

- [Власти используют отзыв зарубежных SSL-сертификатов, чтобы продвинуть сертификаты Минцифры](#) — Теплица социальных технологий
- [Почему все опасаются сертификата от Минцифры](#) — Медуза

### Белые списки

- [Белые списки добрались до Москвы: изучаем механику «отсечки» в 16 килобайт](#) — Хабр
- [Это — всё что вам надо знать о белых списках: как устроены и 6 способов обхода](#) — Хабр
- [Белые списки на домашнем интернете — уже скоро и как подготовиться?](#) — Хабр
- [Мой VPN пережил белые списки. Архитектура из 4 уровней за 265 ₽ в месяц](#) — Хабр
- [russia-mobile-internet-whitelist](#) — списки доменов и подсетей, собранные сообществом
- [Russia: mobile network website whitelist](#) — замеры на Билайне, net4people
- [«Белые списки» сайтов в России](#) — Википедия
- [Белый список цифровых платформ](#) — TAdviser
- [Moscow seeks to limit internet to state-approved websites](#) — The Record

### Клиент и транспорты

- [sing-box](#) — документация
- [URLTest](#) — группа с автовыбором живого профиля